End-to-End CI/CD Pipeline Using Jenkins, Docker, and AWS EKS

Automated Build, Test, Containerization & Kubernetes Deployment

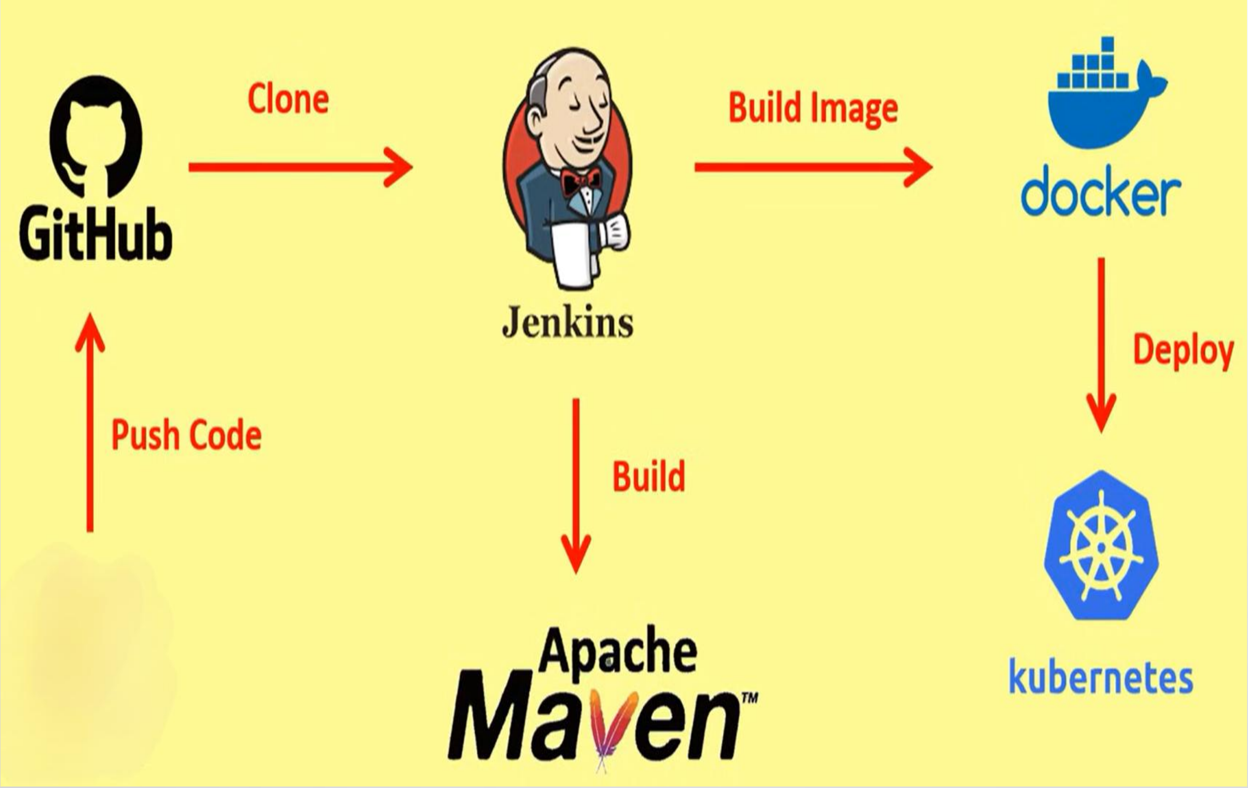
Project Objective

- Create a production-style automated pipeline to reduce manual work, speed up deployments, and ensure reliable, scalable delivery of applications using Jenkins, Docker, and AWS EKS.

Tools & Technologies Used

- Git: Version control for source code and collaboration.
- Maven: Build and package the Java application automatically.
- Jenkins: Automates CI/CD pipeline and triggers builds on code changes.
- Docker: Containerizes the application into portable images.
- Kubernetes (EKS): Orchestrates containers and manages deployment and scaling.

CI/CD Workflow



CI/CD Pipeline Steps

- Clone code from GitHub
- Maven build (compile & package)
- Docker image creation (Dockerfile)
- Push image to Amazon ECR
- Deploy/update application in EKS
- Expose service through Load Balancer

Where This Project Is Used

- Teams that release software frequently and need automation.
- Microservice-based applications running in containers.
- Cloud-native services in e-commerce, banking, healthcare, fintech, and SaaS.
- Environments that require scalable, zero-downtime deployments.

Real-World Example

- **E-commerce website (e.g., Amazon/Flipkart):** Developers push changes daily → Jenkins builds and creates Docker images → ECR stores images → EKS deploys new version using rolling updates → Customers experience no downtime while features are released continuously.

Advantages

- Fully automated deployments and faster release cycles
- Scalable Kubernetes environment with Auto Scaling
- Zero-downtime releases via rolling updates
- Improved reliability and repeatability in deployments

Possible Improvements

Add Monitoring & Alerts:

Integrate tools like Prometheus, Grafana, or CloudWatch to track application and cluster health.

Enable Auto-Rollback:

Automatically revert to the previous stable version if a deployment fails.

Add Automated Testing:

Include unit and integration tests in the pipeline to catch issues early.

CONCLUSION

- This project demonstrates practical DevOps skills—CI/CD automation, containerization, and Kubernetes orchestration—delivering reliable, scalable deployments on AWS.